Gizmonic Cambot: software architecture

Firmware, host libraries, registrar, API and remaining work

Gizmonic Cambot project · Apache-2.0
Plan rev B · November 2026

- 1. Goals
- 2. Repository layout (as built)
- 3. Runtime architecture
- 4. Storage format on the SD card
- 5. Indexing pipeline (on device)
- 6. Model adapters
- 7. MCP tools (on camera)
- 8. HTTP API (v1, as implemented)
- 9. Provisioning and configuration (today)
- 10. Registrar (LAN host)
- 11. Testing
- 12. Milestones
- 13. Contributor rules (`AGENTS.md` summary)

1. Goals

- Record continuously to microSD in a rolling buffer, and never lose recording because of indexing, queries or network trouble.
- Build a semantic index of activity on the device with cloud (or LAN) vision models, within a hard daily budget.
- Answer natural-language questions from a LAN host that keeps an index history, and expose search/inspect tools on the camera through REST and MCP.
- Set up from a fresh board to a camera on the network by following `fixr/SETUP.md`, driven by a person or by a coding agent.
- Support more than one camera board through board-support packages (today: the FORIOT ESP32-S3-CAM pin map in `firmware/main/bringup.c`).

Non-goals for v1: audio, H.264, cloud video storage, face recognition, exposure to the public internet, enterprise Wi-Fi.

2. Repository layout (as built)

One monorepo, `gizmonic-cambot`. Firmware, host tests, registrar, docs and CAD live together. Names below match the tree today; aspirational folders from the October plan that are not present are listed in section 12.

```
gizmonic-cambot/
├── README.md
├── AGENTS.md
├── pyproject.toml          host test package `cambot-host`
├── assets/                 README artwork
├── fixr/
│   ├── SETUP.md            agent-readable setup guide (canonical)
│   ├── README.md
│   ├── profiles/           per-computer JSON + notes (worked examples)
│   ├── schema/             host-profile and camera-registration schemas
│   └── registrar/          LAN web UI, ask, index.sqlite sync
├── firmware/               ESP-IDF 5.x project (`cambot_bringup`)
│   ├── main/               bring-up: Wi-Fi, HTTP, register, secrets.h
│   └── components/
│       ├── recorder/       capture loop, segments, HTTP, MCP, indexer task
│       ├── index/          motion, keyframe, grayscale helpers
│       ├── llm/            OpenAI, Anthropic, JEV body builders + HTTP
│       ├── store/          frame index (idx) helpers
│       └── timebase/       SNTP / sync flags
├── host/
│   ├── cambot_store/       card format, time queries, HTTP client helpers
│   ├── cambot_index/       host-side indexer contract (ported on-device)
│   ├── cambot_models/      OpenAI, Anthropic, JEV adapters
│   └── cambot_mcp/         MCP tool definitions shared with tests
├── hardware/
│   └── cambot-minimum-petg-kit/   PETG case + arms + magnet foot
├── docs/                   white paper, this plan, mechanical spec, build.sh
├── test/
│   ├── host/               Python + C unit tests
│   └── hil/                hardware-in-the-loop helpers
```

Why ESP-IDF rather than Arduino. Cambot needs FreeRTOS task priorities, the IDF HTTP server with several concurrent sockets, SDMMC/FATFS tuning and room for OTA later. The FORIOT pin map, `CAMERA_GRAB_LATEST`, and OV3660 warm-up live in the bring-up firmware. PlatformIO is optional and unused in the current path.

3. Runtime architecture

Task / role	Core	Job
`recorder`	Prefer 1	Grab VGA JPEG, copy to PSRAM, append segment + idx, eviction
`indexer`	Prefer 0	Motion score, keyframe policy, GPT/JEV calls, JSONL index/events
`httpd`	0 (with Wi-Fi)	REST, MCP, frame and clip serving
Registrar (host)	—	Live UI, noul editor, ask, SQLite index history, perf/dev switches

Rule: the recorder never waits on the model. If the SD card stalls, capture drops frames and counts them. If the indexer falls behind, keyframes are dropped. HTTPS to OpenAI or TypeSafe must not hold the camera mutex.

Defaults today: `FRAMESIZE_VGA`, `capture_ms = 200` (~5 fps). High performance mode keeps that pace, flushes the card less often and tells the registrar to poll the live frame every 0.5 s. Flag file: `/sdcard/cambot/high_perf`.

Memory: 8 MB PSRAM holds camera buffers, JPEG copies for indexing and HTTP bodies. **Flash:** single bring-up app image for now; dual OTA slots remain planned.

4. Storage format on the SD card

Root: `/sdcard/cambot` (`CAMBOT_ROOT`). Host libraries and on-device code share the same layout ideas; the camera is authoritative for what is on the card.

```
/sdcard/cambot/
  rec/YYYY-MM-DD/HH/
    YYYY-MM-DDTHH-MM-SSZ.mjpg     concatenated JPEG frames, ~60 s
    YYYY-MM-DDTHH-MM-SSZ.idx      binary frame index (incl. motion milli)
  index/YYYY-MM-DD/HH.jsonl       one keyframe record per line
  events/YYYY-MM-DD.jsonl         merged events
  nouls.json                      JEV questions (+ legacy watch.json read)
  nouls/<name>/…                  example JPEGs (≤ 80 KB each)
  high_perf                       present / content selects perf mode
```

Index record (representative): caption, objects, actions, robot state, motion, segment id, optional JEV `watch` map, optional boxes and alert metadata. Exact fields follow the host indexer contract in `host/cambot_index/pipeline.py` and the on-device writer in `firmware/components/recorder/indexer.c`.

Eviction: when free space is low, delete the oldest hour under `rec/`. Recording continues if indexing is paused by budget or by the registrar's Dev mode.

Card: FAT32, high-endurance recommended. Mount in 1-bit SDMMC on CLK 39, CMD 38, D0 40 (FORIOT ESP32-S3-CAM map).

5. Indexing pipeline (on device)

1. **Motion.** Grayscale, shrink, mean absolute change → 0..1 stored as thousandths in the frame index.
2. **Keyframe.** First frame; then every ~3 s while motion stays above threshold; else every ~60 s.
3. **GPT** (optional). Caption keyframe with `gpt-4.1-mini` into structured JSON.
4. **JEV** (optional). One System One request per keyframe with live frame, noul list, examples, optional box question. Camera calls TypeSafe; the registrar does not.

5. **Persist.** Append JSONL under `index/`; merge events; keep a RAM recent window for MCP `search_index` / ask sync.
6. **Budget.** Daily frame budget pauses the indexer only.

Nouls are yes/no questions with up to four example images, optional box geometry when score ≥ 0.5, optional colors for registrar overlay, and optional HTTPS webhooks that fire on edge into yes or no. Managed via `GET` / `PUT /api/v1/nouls` and example routes under `/api/v1/nouls/...`.

6. Model adapters

Adapter	Where	Notes
OpenAI-compatible	Firmware `llm` + `host/cambot_models`	Captions / ask
Anthropic	Firmware + host	Messages API path exists
JEV / TypeSafe	Firmware + host	Noul scoring on camera

Bring-up keys are compile-time: `CAMBOT_MODEL_KEY`, `CAMBOT_JEV_KEY` in `secrets.h` (gitignored; see `secrets.h.example`). Registrar ask uses `CAMBOT_MODEL_KEY` from the environment or `fixr/run/secrets.env`.

7. MCP tools (on camera)

`POST /mcp` speaks Streamable HTTP / JSON-RPC. Tools implemented on device:

Tool	Purpose
`get_status`	Camera / storage snapshot
`list_recordings`	Minutes that exist in a range
`search_index`	Search the **RAM recent** index window
`read_index_page`	Page deeper history from the card
`inspect_window`	Return frame image content around a time
`make_clip`	Clip URL (`clip.mjpg`)
`get_live_frame`	Current JPEG

Host package `host/cambot_mcp` mirrors the tool list for tests. Full on-board agent loop and `POST /api/v1/ask` are still planned; today **ask** runs in `fixr/registrar/serve.py` against SQLite history synced from the camera.

8. HTTP API (v1, as implemented)

Auth: `Authorization: Bearer <CAMBOT_CAMERA_PASSWORD>` on protected routes. `GET /api/v1/ping` and registration remain the discovery/setup surface.

Method and path	Returns
`GET /api/v1/ping`	Liveness / id
`GET /api/v1/frame.jpg`	Live JPEG
`GET /api/v1/status`	Wi-Fi, time, SD, recording, perf, indexing flags
`GET /api/v1/recordings?from&to`	Segment list
`GET /api/v1/frame/{t}.jpg`	Recorded frame nearest to `t`
`GET /api/v1/clip.mjpg?from&to`	Replay window as MJPEG
`GET /api/v1/index/latest`	Newest index record (registrar caption card)
`GET` / `POST /api/v1/indexing`	GPT indexing switch
`GET` / `POST /api/v1/jev`	JEV indexing switch
`GET` / `PUT /api/v1/nouls`	Noul configuration
`GET` / `PUT /api/v1/nouls/.../examples/...`	Example JPEGs
`GET` / `POST /api/v1/perf`	High performance mode
`POST /mcp`	MCP endpoint
`POST /api/register`	Registrar registration (from camera)

Not implemented yet (still in the October wishlist): live MJPEG stream URL, `clip.avi`, `POST /api/v1/ask`, `POST /api/v1/marks`, `GET /api/v1/events` poll, `GET` / `PATCH /api/v1/config`, `POST /api/v1/ota`, mDNS `_cambot._tcp`, SoftAP / Improv provisioning.

9. Provisioning and configuration (today)

- **Compile-time secrets.** Copy `firmware/main/secrets.h.example` to `secrets.h`. Fill every required field in that template (SSID, Wi-Fi password, camera password, registrar URL, `CAMBOT_TZ`, optional model keys). Never commit the copy.
- **Flash.** ESP-IDF 5.x on the TTL UART (CH340) at 115200. Physical BOOT/RST steps are in `fixr/SETUP.md`.
- **Join and register.** Firmware joins the 2.4 GHz network, serves HTTP, and POSTs `/api/register` to the registrar on a timer.
- **Host secrets.** `CAMBOT_CAMERA_PASSWORD`, `CAMBOT_WIFI_PASSWORD`, `CAMBOT_MODEL_KEY` via environment or `fixr/run/secrets.env`.

Planned replacements: Improv serial, SoftAP portal, NVS-backed config, token rotation via a `cambot` CLI, browser flasher.

10. Registrar (LAN host)

`fixr/registrar/serve.py` is the human UI and the ask path:

- Lists registered cameras; proxies `frame.jpg` and `index/latest` with the bearer token (never printed).
- Edits nouls and example images; toggles GPT indexing, JEV indexing, High performance and Dev mode (idle → turn indexing off).
- Draws noul boxes on the live image; charts recent scores.
- Syncs index lines into `fixr/run/index.sqlite` and answers questions with gpt-4.1-mini from that history so the camera stays reachable.

HIL helpers: `test/hil/jev_watch.py`, `perf_check.py`, `record_check.py`.

11. Testing

- **Host tests:** `pytest` over `host/cambot_*` and registrar UX (`test/test_registrar_ux.py`, model adapters, store/index/mcp).
- **C helpers:** motion, idx and LLM body builders under `test/host/c/`.
- **HIL:** manual scripts against a flashed board and a running registrar.
- **CI / eval / web flasher:** not wired yet.

12. Milestones

Milestone	Scope	Status
M0	IDF project, FORIOT pins, camera, SD	**Done**
M1	Recorder, eviction, REST frame/status/recordings/clips	**Done**
M2	Improv, SoftAP, `cambot` CLI, web flasher	**Not started**
M3	Motion, on-device GPT + JEV, nouls, registrar live UI	**Done**
M4a	MCP on camera; registrar ask + SQLite history	**Done**
M4b	On-board `/ask`, marks, richer index/events HTTP	Planned
M5	OTA rollback, encrypted builds, second BSP	Planned

Milestone	Scope	Status
Later	Embeddings, multi-camera hub	Planned

13. Contributor rules (`AGENTS.md` summary)

- Never commit secrets. `firmware/main/secrets.h` , `.env` , API key files and `fixr/run/` stay gitignored.
- Prefer environment variables or hidden prompts for Wi-Fi password, camera password and model keys. Never print them.
- Recording comes before everything else. A change that blocks the recorder task is rejected.
- Printable kit changes go through the generators, exports and `manifest.json` together.
- Ask before formatting an SD card, erasing flash or rotating a token.
- Until M2 ships, do not invent `cambot` CLI commands; follow `fixr/SETUP.md` .